AMD EMBEDDED DEVELOPMENT FRAMEWORK

Migrating a Vivado design to EDF with agentic AI

Agent reads and cross-checks. Tools validate. Engineer decides.

1
Interrogate
Real IP config and address map

2
Rebuild PL
Scripted design → bitstream + XSA

3
Firmware
Overlay + shell.json for dfx-mgr

4
Image
Own layer, BitBake, inspect

5
Bring-up
Verify each boundary

Coding agent

Reads sources, docs and bindings; drafts Tcl, overlays, recipes.

Vivado MCP

Live Tcl: query IP, validate, implement, report.

Engineer

Runs `bitbake` and the board; approves every change.

Know the design, then rebuild it for Linux

1 · Interrogate via MCP

- Extract IP config, widths, clocks, IRQs, address map
- Trust the project, not the spec
- Flag what won't port: missing IP, fixed buffers

```
get_bd_addr_segs -hier
report_property [get_bd_cells -hier *]
```

2 · Rebuild as Tcl

- Keep the original project as reference
- Prefer IP with upstream Linux drivers
- Iterate on `validate_bd_design`; fix causes

```
report_timing_summary ; report_drc
write_hw_platform -fixed -include_bit hw.xsa
```

GATE Setup/hold slack ≥ 0 · zero DRC errors · XSA matches address map

Describe the PL to Linux

Device-tree overlay

- Agent drafts from address map, AMD reference overlays, kernel bindings
- Check `reg`, `interrupts`, clocks per node
- Compile early

```
dtc -@ -I dts -O dtb -o app.dtbo app.dtso
```

Firmware recipe

- Bundle `.bit.bin` + `.dtbo` + `shell.json`
- Install to `/lib/firmware/xilinx/app`

```
DEPENDS = "dtc-native"        # build
RDEPENDS:${PN} = "dfx-mgr"    # target
```

ARTIFACT One firmware package that `dfx-mgr` can load

Build on EDF; inspect before flashing

Workspace

- AMD EDF build container
- Bind-mount a host directory
- `repo` sync the EDF manifest

Your layer

- Image, app and firmware recipes
- systemd units
- Kernel `.cfg` fragment
- One consistent `MACHINE`

Inspect

- Agent checks manifest and packages
- Loop-mount the rootfs
- Confirm firmware, units, tools

```
MACHINE=kria-zynqmp-generic bitbake app-image
```

RULE The build output is the product: never hand-patch the image or board

Verify one boundary at a time

Checks

```
xmutil listapps          # PL loaded
dmesg | grep -i probe    # drivers bound
media-ctl -p             # devices
journalctl -u app        # application
```

- Test sources first, real inputs last
- Cold boot, reboot, unplug/replug

Working with the agent

- Paste real output, not summaries
- One bounded check at a time
- Diagnose on board, fix in recipes
- Ask what consumes each setting
- Tools have the final say

DONE A clean build boots on fresh media and works end-to-end, no manual steps